TGS · Taller Global de Servicios

Inventario de scripts del proyecto y guia de operacion

> **Proyecto:** TGS - Taller Global de Servicios (Blue Tech Uruguay)
> **Servidor:** maquina virtual `basedatos` - Ubuntu 26.04 - Docker (Apache + PHP 8.4 / MySQL 8.4)
> **Website publico:** `https://tgs.bluetech.workers.dev` (dominio fijo, gratuito)
> **Documento:** generado a partir del codigo fuente del repositorio `ttugol909674-cpu/index`

1. scripts/ — Operacion del servidor (se ejecutan en la VM)

Script	Que hace	Ejemplo
`_entorno.sh`	Libreria base. **No se ejecuta**: se carga con `source`. Mezcla `.env` (global, versionado) y `.env.local` (local, con las claves reales) y los exporta al entorno, aplicando la precedencia del proyecto.	`source scripts/_entorno.sh` `tgs_cargar_entorno`
`instalar.sh`	Instalacion completa: verifica Docker, genera el `.env.local` con claves nuevas, construye la imagen y levanta los contenedores.	`bash scripts/instalar.sh`
`estado.sh`	El "esta todo bien?" rapido: contenedores, base de datos y sitio web. Es el comando para comprobar, antes y despues de cada paso del video, que el servidor esta en pie.	`bash scripts/estado.sh`
`operador.sh`	Script operador del centro de computos. Reune en un solo comando las tareas de administracion del servidor: `servicios`, `red`, `procesos`, `usuarios`, `respaldos`, `tunel`, entre otras.	`bash scripts/operador.sh servicios` `bash scripts/operador.sh red`
`base.sh`	Abre la consola de MySQL de la base del proyecto. La clave se lee sola del `.env.local`, asi que no hay que escribirla.	`bash scripts/base.sh`
`herramientas.sh`	Levanta o detiene las herramientas opcionales (phpMyAdmin). Carga el entorno del proyecto antes de llamar a Docker Compose, que solo lee `.env`.	`bash scripts/herramientas.sh up ... down` / `... status`
`respaldar.sh`	Respaldo de la base de datos: saca un `mysqldump` del contenedor MySQL y lo guarda	`bash scripts/respaldar.sh`

	en `backups/tgs_<fecha>.sql`. Gira los respaldos viejos y conserva los 5 mas recientes.	
`restaurar.sh`	Restauracion desde un respaldo: vacia las tablas del contenedor y vuelve a cargar el `.sql` elegido (o el mas reciente).	`bash scripts/restaurar.sh`
`caida.sh`	Simulacion de la caida del servidor, para la defensa. Sin argumentos borra contenedores **y** volumen (perdida total de datos); con `--suave` solo detiene y conserva la informacion.	`bash scripts/caida.sh --suave`
`crear_roles_bd.sh`	Crea los tres perfiles de acceso a MySQL (`tgs`, `tgs_admin`, `tgs_lectura`) con los permisos justos. Responde al requisito de "Seguridad y Administracion: creacion de roles de usuario en el gestor de base de datos para acceder a MySQL de forma segura".	`bash scripts/crear_roles_bd.sh`
`firewall.sh`	Configura el firewall con iptables: solo la red local (192.168.1.0/24) entra a los puertos del servidor, y deja un servicio del sistema que restaura las reglas al arrancar.	`sudo bash scripts/firewall.sh aplicar` `... mostrar` / `... revertir`
`tunel.sh`	Publica el sitio en internet con Cloudflare Tunnel (URL temporal). El servidor se conecta hacia afuera, asi que no hay que abrir puertos en el router.	`bash scripts/tunel.sh iniciar` `... detener` / `... estado`
`tunel_nombrado.sh`	Tunel **con nombre**: URL permanente y gratuita. Se crea una sola vez en la cuenta de Cloudflare y conserva su identificador para siempre.	`bash scripts/tunel_nombrado.sh autorizar` `crear` / `iniciar` / `estado`
`tunel_vigilante.sh`	Mantiene al dia el archivo con la URL publica (archivo `.tunel.url`) y escribe la direccion vigente en `url-publica.txt`, consultable desde el navegador.	`bash scripts/tunel_vigilante.sh`
`actualizar.sh`	Sincroniza el codigo con GitHub. Se ejecuta cada 2 minutos desde el cron. Solo hace fast-forward y nunca pisa cambios locales reales.	`bash scripts/actualizar.sh`
`publicar-url.sh`	Le avisa al Worker de Cloudflare cual es la URL vigente del tunel, para que la direccion publica (workers.dev) no cambie nunca aunque el tunel se reinicie.	`bash scripts/publicar-url.sh`

2. scripts/ — Para el equipo de desarrollo (Windows)

Script	Que hace	Ejemplo
`sync-vm.ps1`	Vigila la carpeta del proyecto y copia a la maquina virtual los archivos que se modifican en Visual Studio Code (cada ~15 segundos). Normaliza los finales de linea a LF antes de copiarlos, porque el repositorio los exige asi.	`powershell -File scripts/sync-vm.ps1`
`sync-vm-vigilante.ps1`	Revisa cada 60 segundos que el sincronizador siga vivo y lo vuelve a levantar si se cae (reinicio del equipo, suspension, cierre de sesion).	Se arranca solo con la sesion de Windows
`sync-vm-iniciar.ps1`	Lanzador: si no hay ninguna instancia del sincronizador viva, arranca una. Pensado para las tareas programadas de Windows.	`powershell -File scripts/sync-vm-iniciar.ps1`

> **Flujo de trabajo completo:** se edita en VS Code → el sincronizador lleva el cambio a la VM (15 s) → el contenedor lo ve al instante porque el codigo esta montado en caliente → la web publica queda actualizada. La base de datos es una sola: la de la VM, a la que apuntan tanto el desarrollo de Windows como la web publicada.

3. tests/ — Pruebas automaticas

Script	Que hace
`run_suite.sh`	Corre **las 28 pruebas** seguidas, restaurando los datos de demostracion antes de cada una para que no se pisen entre si. Copia los tests a `/tmp` (nunca a la carpeta del proyecto) y avisa si falta alguno, para no reportar un OK falso.
`tgs_test_lib.sh`	Libreria compartida: inicio de sesion con token CSRF, obtencion de tokens de formularios y cookies en `/tmp`.
`credenciales.local.sh`	Las claves de las cuentas de demostracion. **No se versiona** (esta en el `.gitignore`).
`credenciales.ejemplo.sh`	Plantilla para copiar y completar con las claves reales.
`restore_demo_full.sh`	Deja la base con los datos de demostracion: 2 reservas (una pendiente y pagada, otra finalizada con factura), presupuestos, pagos y avisos.
`smoke.sh`	Prueba de humo: comprueba que el sitio responde.
`lint_vm.sh`	Revisa que la maquina virtual tenga lo que debe: Docker, puertos, tareas de cron y reglas de firewall.

Las 26 pruebas funcionales

Prueba	Que verifica
`test_csrf.sh`	Un POST sin token anti-CSRF no crea nada; con token, si. Ademas la cookie de sesion y el cierre de sesion.
`test_idiomas.sh`	Las 23 pantallas muestran el texto del idioma pedido, sin fugas de un idioma al otro, el selector ES/EN en todas, la persistencia al navegar y tras iniciar sesion, y que un idioma invalido caiga en espanol.
`test_alta.sh`	Alta de usuario desde el panel: rol, correo y telefono validos, duplicados, clave corta y clave generada automaticamente.
`test_baja.sh`	Baja de cuenta: pide la clave, borra vehiculos, reservas, pagos, presupuestos y avisos, y no borra con clave incorrecta ni sin CSRF.
`test_editar.sh`	Edicion de datos de usuario y de rol.
`test_perfil.sh`	Cambio de datos del perfil y de la contrasena.
`test_agenda.sh`	Agenda del gestor: navegacion por fecha y listado de turnos.

`test_detalle.sh`	Detalle de un turno: datos, cobros, presupuestos, avisos e historial.
`test_repro.sh`	Reprogramacion de un turno con actualizacion del historial de cambios.
`test_disp.sh`	Disponibilidad de horarios por sede, capacidad y franja.
`test_pagos.sh`, `test_pagos2.sh`	Reporte de pago del cliente, aprobacion, rechazo y anulacion por parte del taller.
`test_a_pagar.sh`	Calculo del saldo a pagar con saldo prepago.
`test_pago_cliente.sh`	Pago del cliente con el saldo de la cuenta.
`test_presupuestos.sh`	Creacion de presupuestos y sus items, aceptacion y rechazo por el cliente.
`test_notificaciones.sh`	Avisos al cliente y su registro en la tabla de notificaciones.
`test_matriz.sh`, `test_permisos.sh`	Matriz de permisos por rol y su edicion en caliente.
`test_ownership.sh`	Un cliente no puede ver ni tocar datos de otro cliente.
`test_alta_taller.php` → `test_alta_taller.sh`	Alta de una sede con su capacidad, horarios, marcas y servicios.
`test_nav.sh`, `test_modal.sh`	Navegacion y ventanas modales.
`test_filtros.sh`, `test_filtros2.sh`, `test_filtros3.sh`	Busqueda y filtros del panel de usuarios.
`test_fallback.sh`	Recursos inexistentes devuelven la pagina de error 403 en vez de un error del servidor.
`test_smtp.sh`	Envio de avisos: comprueba que el aviso queda registrado con su estado de transporte.
`test_panel.sh`, `test_gaps.sh`	Revision general de paneles y de casos borde.

> **Resultado actual de la suite:** `SUITE_FALLOS=0` - las 28 pruebas se ejecutan de verdad y pasan todas.

4. Comandos para la defensa

```
bash scripts/estado.sh # el servidor esta en pie
```

```
bash scripts/operador.sh servicios # control de servicios (Docker, SSH, cron)
```

```
bash scripts/operador.sh red # red, puertos y rutas
```

```
bash scripts/crear_roles_bd.sh # roles de MySQL con permisos justos
```

```
bash scripts/caida.sh --suave # simular la caida y recuperar
```

```
bash scripts/respaldar.sh # respaldo de la base
```

```
bash scripts/restaurar.sh # restauracion desde el respaldo
```

```
bash scripts/base.sh # consola de MySQL
```

```
bash tests/run_suite.sh # las 28 pruebas
```

Direcciones

Que	Direccion
Web publica (fija)	`https://tgs.bluetech.workers.dev`
Sitio en la red local	`http://192.168.1.12:8080/`
Desarrollo en Windows	`http://localhost:8012/tgs/`
phpMyAdmin (base de datos)	`http://192.168.1.12:8081/` - sin usuario ni contrasena
phpMyAdmin desde la PC	`http://localhost:8012/phpmyadmin/` - elegir "TGS - Base de datos de la maquina virtual"
Acceso a MySQL	`192.168.1.12:3307` - base `bluetech_base` - usuario `tgs`

Datos de las cuentas de demostracion

Rol	Correo / contrasena
Administrador	`admin@bluetech.com` / `Admin123!`
Gestor	`santiiuyy12@gmail.com` / `Gestor123!`
Cliente	`carlos.gomez@gmail.com` - `mariana.silva@gmail.com` - `bysantiiuyy@gmail.com` / `Cliente123!`

> **Nota:** las contrasenas de arriba coinciden con los hash bcrypt de `db/seed.sql`. Como el sitio esta publicado en internet, conviene cambiarlas antes de la defensa y actualizar el `seed.sql` con los hash nuevos (en el seed solo debe circular el hash, nunca el texto plano).

> **Recordatorio tecnico:** `docker compose` solo lee el archivo `.env`, nunca el `.env.local` (donde estan las claves reales de la base). Por eso las herramientas se lanzan con `scripts/herramientas.sh`, que carga el entorno del

> proyecto antes de llamar a Compose.
